

Anexo 05 — Entrenamiento PPO de la cabeza

Archivo: Scripts/train_ppo.py

Para que sirvio:

Script de entrenamiento de la policy de seguimiento. Incluye el curriculum del cono de vision (ampliacion del rango de azimuth) y el shaping anti-inclinacion que logro que la cabeza use el giro (yaw) en vez de inclinar la base (resultado v14c).

```
1 | """Entrenamiento PPO sobre DUMHeadTrackingEnv usando Stable-Baselines3.
2 |
3 | Uso:
4 |     # Smoke training: 20k steps single-env para validar pipeline
5 |     python train_ppo.py --smoke
6 |     # Training real: 2M steps con 8 envs paralelos
7 |     python train_ppo.py --steps 2000000 --n-envs 8
8 |     # Resumir desde checkpoint
9 |     python train_ppo.py --steps 2000000 --resume runs/ppo_dum_xxx/checkpoint.zip
10 | """
11 |
12 | import argparse
13 | import sys
14 | import time
15 | from pathlib import Path
16 | from datetime import datetime, timedelta
17 |
18 | sys.path.insert(0, str(Path(__file__).resolve().parent))
19 |
20 | # Limitar threads de PyTorch para no pelear con los workers MuJoCo
21 | # (recomendado por el experto: 16 logical CPUs / 8 fisicos -> torch en 2, libra 6 para envs)
22 | import torch
23 | torch.set_num_threads(2)
24 |
25 | import numpy as np
26 | from stable_baselines3 import PPO
27 | from stable_baselines3.common.vec_env import DummyVecEnv, SubprocVecEnv
28 | from stable_baselines3.common.callbacks import CheckpointCallback, BaseCallback
29 | from stable_baselines3.common.monitor import Monitor
30 |
31 | from rl_env import DUMHeadTrackingEnv
32 |
33 | RUNS_DIR = Path(__file__).resolve().parents[1] / "runs"
34 |
35 |
36 | class HeadCurriculumV14Callback(BaseCallback):
37 |     """v14_head: doble curriculum durante el resume desde v13.
38 |
39 |     1. Cone ramp (0 → cone_ramp_steps): ampliacion lineal del cono de target
40 |     desde (75°,45°) hasta (150°,50°). v13 entreno con (75°,45°) y nunca vio
41 |     targets mas alla – entonces necesita exposicion gradual al area extendida.
42 |     2. Ent_coef step (a ent_switch_steps): empieza alto (0.03) para forzar
43 |     exploracion en dimensiones del action space (esp. HeadRot) que v13 dejo
44 |     quietas, baja a 0.005 cuando ya tuvo tiempo de salir del local optimum.
45 |     """
46 |     def __init__(self, total_timesteps: int,
47 |                  cone_start_az_deg: float = 75.0, cone_end_az_deg: float = 150.0,
48 |                  cone_start_el_deg: float = 45.0, cone_end_el_deg: float = 50.0,
49 |                  cone_ramp_steps: int = 7_000_000,
50 |                  ent_high: float = 0.03, ent_low: float = 0.005,
51 |                  ent_switch_steps: int = 5_000_000,
52 |                  log_every: int = 1_000_000, verbose: int = 0):
53 |         super().__init__(verbose)
54 |         self.total = int(total_timesteps)
55 |         self.cone_start_az = float(np.deg2rad(cone_start_az_deg))
56 |         self.cone_end_az = float(np.deg2rad(cone_end_az_deg))
57 |         self.cone_start_el = float(np.deg2rad(cone_start_el_deg))
58 |         self.cone_end_el = float(np.deg2rad(cone_end_el_deg))
59 |         self.cone_ramp_steps = int(cone_ramp_steps)
60 |         self.ent_high = float(ent_high)
61 |         self.ent_low = float(ent_low)
62 |         self.ent_switch_steps = int(ent_switch_steps)
63 |         self.log_every = int(log_every)
64 |         self._last_logged = 0
65 |         self._last_ent = None
66 |         self._switched_ent = False
67 |
68 |     def _on_step(self) -> bool:
69 |         # Cone ramp
70 |         if self.num_timesteps < self.cone_ramp_steps:
71 |             t = self.num_timesteps / self.cone_ramp_steps
72 |             cone_az = self.cone_start_az + t * (self.cone_end_az - self.cone_start_az)
73 |             cone_el = self.cone_start_el + t * (self.cone_end_el - self.cone_start_el)
```

```

74 |         else:
75 |             cone_az = self.cone_end_az
76 |             cone_el = self.cone_end_el
77 |         try:
78 |             self.training_env.env_method("set_cone_limits", cone_az, cone_el)
79 |         except Exception:
80 |             pass
81 |
82 |         # ent_coef step transition
83 |         if self.num_timesteps < self.ent_switch_steps:
84 |             target_ent = self.ent_high
85 |         else:
86 |             target_ent = self.ent_low
87 |         if self._last_ent != target_ent:
88 |             self.model.ent_coef = target_ent
89 |             self._last_ent = target_ent
90 |             if self._switched_ent or self.num_timesteps > 1000:
91 |                 from datetime import datetime
92 |                 stamp = datetime.now().strftime("%H:%M:%S")
93 |                 print(f"[v14_head {stamp}] ent_coef set to {target_ent} @ steps={self.num_timesteps:}",
94 |                       flush=True)
95 |             self._switched_ent = True
96 |
97 |         # Log periodico
98 |         if self.verbose and (self.num_timesteps - self._last_logged) >= self.log_every:
99 |             from datetime import datetime
100 |             stamp = datetime.now().strftime("%H:%M:%S")
101 |             print(f"[v14_head {stamp}] steps={self.num_timesteps:}, "
102 |                   f"cone={np.rad2deg(cone_az):.0f}°az/{np.rad2deg(cone_el):.0f}°el "
103 |                   f"ent_coef={self.model.ent_coef:.4f}",
104 |                   flush=True)
105 |             self._last_logged = self.num_timesteps
106 |         return True
107 |
108 |
109 | def make_env(seed=0, log_dir=None, cone_az_deg=None, cone_el_deg=None,
110 |             w_headrot_guidance=0.0, w_headbase_tilt_penalty=0.0):
111 |     def _init():
112 |         rw = {}
113 |         if w_headrot_guidance > 0.0:
114 |             rw["w_headrot_guidance"] = w_headrot_guidance
115 |         if w_headbase_tilt_penalty > 0.0:
116 |             rw["w_headbase_tilt_penalty"] = w_headbase_tilt_penalty
117 |         env = DUMHeadTrackingEnv(
118 |             reward_weights=rw if rw else None,
119 |             cone_az_deg=cone_az_deg,
120 |             cone_el_deg=cone_el_deg,
121 |         )
122 |         env.reset(seed=seed)
123 |         if log_dir is not None:
124 |             env = Monitor(env, str(log_dir / f"monitor_{seed}.csv"))
125 |         return env
126 |     return _init
127 |
128 |
129 | def main():
130 |     p = argparse.ArgumentParser()
131 |     p.add_argument("--steps", type=int, default=2_000_000)
132 |     p.add_argument("--n-envs", type=int, default=8)
133 |     p.add_argument("--smoke", action="store_true",
134 |                    help="Override: 20k steps, 1 env, para validar pipeline")
135 |     p.add_argument("--resume", type=str, default=None)
136 |     p.add_argument("--name", type=str, default=None)
137 |     p.add_argument("--ent-coef", type=float, default=0.0,
138 |                    help="Coeficiente de entropia en la loss de PP0. 0 = sin exploracion forzada. "
139 |                          "Subir a 0.005-0.01 si la policy converge prematuramente a local optimum.")
140 |     p.add_argument("--learning-rate", type=float, default=3e-4)
141 |     p.add_argument("--n-steps", type=int, default=2048,
142 |                    help="Rollout por env antes de cada update. Total = n_steps * n_envs.")
143 |     p.add_argument("--batch-size", type=int, default=64,
144 |                    help="Minibatch size para SGD. Subir con mas envs (256 si n_envs>=8).")
145 |     p.add_argument("--n-epochs", type=int, default=10,
146 |                    help="Numero de passes sobre el rollout por update.")
147 |     p.add_argument("--net-arch", type=str, default="64,64",
148 |                    help="Arquitectura del MLP (pi y vf). Ejemplo: '128,128' o '256,256,128'.")
149 |     p.add_argument("--head-v14-curriculum", action="store_true",
150 |                    help="Activa curriculum v14_head: cone ramp 75°-150° az + ent_coef step 0.03-0.005")
151 |     p.add_argument("--v14-cone-ramp-steps", type=int, default=7_000_000)
152 |     p.add_argument("--v14-ent-switch-steps", type=int, default=5_000_000)
153 |     p.add_argument("--v14-cone-end-az", type=float, default=150.0,
154 |                    help="Cono azimuth final en grados (default 150°, max fisico 160°)")
155 |     p.add_argument("--v14-cone-end-el", type=float, default=50.0,
156 |                    help="Cono elevation final en grados (default 50°)")
157 |     # v14b: training from scratch con cono fijo + reward shaping
158 |     p.add_argument("--cone-az-deg", type=float, default=None,

```

```

159 |         help="(v14b) Cono azimuth en grados, fijo desde el inicio")
160 |     p.add_argument("--cone-el-deg", type=float, default=None,
161 |         help="(v14b) Cono elevation en grados, fijo desde el inicio")
162 |     p.add_argument("--w-headrot-guidance", type=float, default=0.0,
163 |         help="(v14b) Penalty weight: cuando target_az > 17°, penaliza HeadRot_qpos lejos del target_az.
v14c usa 2.0.")
164 |     p.add_argument("--w-headbase-tilt-penalty", type=float, default=0.0,
165 |         help="(v14c) Penalty directo a |HeadBase_qpos| cuando target_az > 30°. Fuerza a NO usar HeadBase
para compensar yaw. v14c usa 1.5.")
166 |     args = p.parse_args()
167 |
168 |     if args.smoke:
169 |         args.steps = 20_000
170 |         args.n_envs = 1
171 |
172 |     timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
173 |     name = args.name or f"ppo_dum_{timestamp}"
174 |     run_dir = RUNS_DIR / name
175 |     run_dir.mkdir(parents=True, exist_ok=True)
176 |     print(f"Run dir: {run_dir}")
177 |
178 |     # Crear envs paralelos
179 |     env_kwargs = dict(
180 |         cone_az_deg=args.cone_az_deg,
181 |         cone_el_deg=args.cone_el_deg,
182 |         w_headrot_guidance=args.w_headrot_guidance,
183 |         w_headbase_tilt_penalty=args.w_headbase_tilt_penalty,
184 |     )
185 |     if args.n_envs == 1:
186 |         env = DummyVecEnv([make_env(seed=0, log_dir=run_dir, **env_kwargs)])
187 |     else:
188 |         env = SubprocVecEnv([make_env(seed=i, log_dir=run_dir, **env_kwargs) for i in range(args.n_envs)])
189 |
190 |     # Hiperparams PPO tipo MuJoCo continuous control
191 |     if args.resume:
192 |         print(f"Resuming from {args.resume}")
193 |         model = PPO.load(args.resume, env=env)
194 |         from_scratch = False
195 |     else:
196 |         # Parse net-arch "a,b,c" -> [a,b,c]
197 |         net_arch = [int(x) for x in args.net_arch.split(",") if x.strip()]
198 |         model = PPO(
199 |             "MlpPolicy",
200 |             env,
201 |             verbose=1,
202 |             tensorboard_log=str(run_dir / "tb"),
203 |             learning_rate=args.learning_rate,
204 |             n_steps=args.n_steps,
205 |             batch_size=args.batch_size,
206 |             n_epochs=args.n_epochs,
207 |             gamma=0.99,
208 |             gae_lambda=0.95,
209 |             clip_range=0.2,
210 |             ent_coef=args.ent_coef,
211 |             vf_coef=0.5,
212 |             policy_kwargs=dict(net_arch=dict(pi=net_arch, vf=net_arch)),
213 |         )
214 |         from_scratch = True
215 |
216 |     checkpoint_cb = CheckpointCallback(
217 |         save_freq=max(args.steps // 10 // args.n_envs, 1),
218 |         save_path=str(run_dir),
219 |         name_prefix="checkpoint",
220 |     )
221 |     callbacks = [checkpoint_cb]
222 |     if args.head_v14_curriculum:
223 |         callbacks.append(HeadCurriculumV14Callback(
224 |             total_timesteps=args.steps,
225 |             cone_end_az_deg=args.v14_cone_end_az,
226 |             cone_end_el_deg=args.v14_cone_end_el,
227 |             cone_ramp_steps=args.v14_cone_ramp_steps,
228 |             ent_switch_steps=args.v14_ent_switch_steps,
229 |             verbose=1,
230 |         ))
231 |
232 |     # === Banner de inicio ===
233 |     started_at = datetime.now()
234 |     print("=" * 70)
235 |     print(f"INICIO:          {started_at.strftime('%Y-%m-%d %H:%M:%S')}")
236 |     print(f"Run:              {name}")
237 |     print(f"Modo:              {'FROM SCRATCH (red nueva, pesos random)' if from_scratch else 'RESUMED'}")
238 |     print(f"Steps:             {args.steps:,}")
239 |     print(f"N envs:            {args.n_envs}")
240 |     print(f"n_steps/env:       {args.n_steps} -> rollout total = {args.n_steps * args.n_envs}")
241 |     print(f"batch_size:        {args.batch_size}")

```

```

242 |     print(f"n_epochs:      {args.n_epochs}")
243 |     print(f"net_arch:      {args.net_arch}")
244 |     print(f"learning_rate: {args.learning_rate}")
245 |     print(f"ent_coef:      {args.ent_coef}")
246 |     print(f"torch threads: {torch.get_num_threads()}")
247 |     print("=" * 70, flush=True)
248 |
249 |     t0 = time.perf_counter()
250 |     model.learn(
251 |         total_timesteps=args.steps,
252 |         callback=callbacks,
253 |         progress_bar=True,
254 |     )
255 |     elapsed = time.perf_counter() - t0
256 |
257 |     final_path = run_dir / "final.zip"
258 |     model.save(str(final_path))
259 |     finished_at = datetime.now()
260 |
261 |     # === Banner de fin ===
262 |     print("=" * 70)
263 |     print(f"INICIO:      {started_at.strftime('%Y-%m-%d %H:%M:%S')}")
264 |     print(f"FIN:        {finished_at.strftime('%Y-%m-%d %H:%M:%S')}")
265 |     print(f"DURACION:    {timedelta(seconds=int(elapsed))} ({elapsed:.1f} s)")
266 |     print(f"Modelo final: {final_path}")
267 |     print("=" * 70)
268 |
269 |
270 | if __name__ == "__main__":
271 |     main()
272 |

```